# Lab 1 — Requirements Engineering & UML Use-Case Modelling

**Problem Statement #42 — Internal Microservice Catalog & Health Portal**

*Likith Kumar K M | PES University, Dept. of CSE(AIML)*

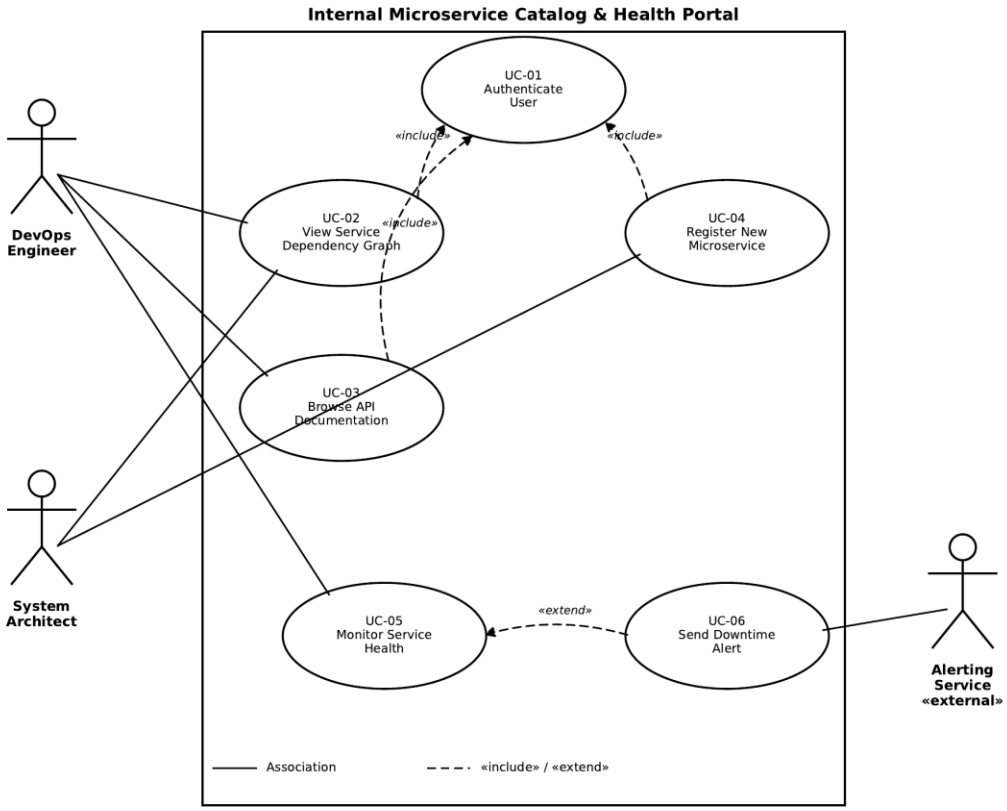
## 1. Requirements Table

| Req ID | Type | Description | Priority | Acceptance Criteria | Rationale |
|---|---|---|---|---|---|
| **FR-001** | Functional | The system shall ping each registered microservice's health endpoint every 30 seconds and keep a rolling 24-hour uptime percentage for it. | High | Pass: after 3 failed probes in a row, the service gets flagged down and an alert goes out. Fail: a service that's actually unreachable still shows up as healthy. | This is the backbone of the whole portal - nothing else in the system works if we don't know a service's real status. |
| **FR-002** | Functional | The system shall let a System Architect open an interactive graph showing which microservices call which other ones. | High | Pass: clicking a service highlights what it depends on and what depends on it, in under a second. Fail: the graph won't load, or an edge is missing/wrong for a service that's already registered. | This is basically the whole point of the problem statement - you can't do impact analysis during an incident if you don't know what's connected to what. |
| **FR-003** | Functional | The system shall let a System Architect register a new microservice - name, health-check URL, owning team, and its known dependencies. | Medium | Pass: submitting a valid form gets the service into the catalog and the graph within about 10 seconds. Fail: it lets you save a broken or duplicate endpoint URL without complaint. | Nothing gets monitored or documented until it's onboarded, so this has to exist - though it's less urgent day-to-day than the monitoring and alerting side. |
| **FR-004** | Functional | The system shall pull in and display each service's OpenAPI/Swagger documentation on its catalog page. | Medium | Pass: opening a service's page shows its current API spec - endpoints, methods, schemas. Fail: the docs shown are stale, or the link is dead. | Saves engineers from digging through repos to find an API contract, which is one of the things the problem statement calls out directly. |
| **FR-005** | Functional | The system shall send a downtime alert to the on-call DevOps Engineer through the configured Alerting Service (email/Slack, whatever's set up) once a service hits 3 failed probes in a row. | High | Pass: the on-call engineer gets notified within 5 seconds of the 3rd failure, and the message names the service and when it went down. Fail: no alert fires, or one fires before 3 consecutive failures have actually happened. | Detecting downtime is pointless if nobody finds out about it - this is what actually closes the loop. |
| **NFR-001** | Nonfunctional | The dependency-graph viewer has to handle up to 200 services without lagging, and stay within target latency and security limits even under simulated peak load. | High | Pass: with 200 nodes under simulated peak traffic, first render stays under 2s and clicking a node responds in under 200ms, all over an authenticated TLS session. Fail: the graph chokes, or works without enforcing TLS/session checks. | A mid-size org can easily have hundreds of services - if the graph isn't fast at that scale, people will just stop using it. |
| **NFR-002** | Nonfunctional | The system shall only show the catalog, graph, and health dashboards to logged-in users, and enforce role-based access so System Architects can add/remove services while DevOps Engineers can only view and monitor. | High | Pass: hitting any of those pages while logged out gets you bounced (401 / redirect to login); a DevOps Engineer who tries to delete a service gets a 403. Fail: any of that data is reachable without logging in, or a DevOps Engineer manages to delete or edit a service record. | Endpoint URLs and service topology are basically a map of the internal infrastructure - that shouldn't be open to anyone who wanders in. |

**Actors: DevOps Engineer, System Architect, and an external Alerting Service.**

*Note: FR-001 and NFR-001 came straight from the problem statement as sample requirements - I just tightened the acceptance criteria so they're properly testable. FR-002 through FR-005 and NFR-002 I wrote myself, working off the rest of the scenario (dependency mapping, documentation, onboarding, and access control).*

# 2. UML Use-Case Diagram




*Three actors - DevOps Engineer, System Architect, and an external Alerting Service - along with an «include» relationship (everything routes through Authenticate User) and an «extend» relationship (Send Downtime Alert only fires conditionally, off of Monitor Service Health).*

# 3. Use-Case Flow Specification

## Use Case: UC-05 — Monitor Service Health

Primary Actor: the Health-Check Scheduler - it's an automated job, but it's effectively acting on the DevOps Engineer's behalf.

Supporting Actors: DevOps Engineer (watches the dashboard), Alerting Service (comes in through the extension, UC-06).

Related Requirements: FR-001, FR-005

## Preconditions

- The service is already registered in the catalog with a working health-check URL (i.e., UC-04 has already happened for it).
- Monitoring is switched on for that service, and the scheduler itself is up and running.

## Postconditions

- If it goes well: the 24-hour uptime number is refreshed, and the dashboard shows the service's current status.
- If it doesn't: the service is marked Down, UC-06 has already fired off an alert, and there's a timestamped log entry for whoever ends up looking into it.

## Main Success Scenario

1. Every 30 seconds, the scheduler kicks off a health probe for a registered service.
2. It sends an HTTP request to that service's health-check endpoint.
3. The service answers back with a 200 OK before the timeout runs out.
4. That gets logged as a successful probe, timestamped.
5. The rolling 24-hour availability number gets recalculated.
6. The dashboard updates to show the service as Healthy.
7. And that's it - use case ends there.

## Alternate Flow A — Service Fails Health Probes (extends to UC-06: Send Downtime Alert)

3a. Step 3 doesn't go as planned - either nothing comes back before the timeout, or the response isn't a 2xx.

3a1. The probe gets logged as a failure, with a timestamp.

3a2. The system checks how many failures in a row this service has had.

3a3. Fewer than 3? Status goes to Degraded, and it just waits for the next scheduled probe - back to step 1.

3a4. Exactly 3 in a row? The service gets marked Down, which kicks off UC-06 - the on-call DevOps Engineer gets notified through the Alerting Service.

3a5. Once it responds successfully again, status flips back to Healthy, the failure count resets, and it's back to the main flow.